



A changer une fois l'horloge du projet assembler par la photo de cette dernière

TPI : Horloge à LED avec Capteur

Documentation du projet

Table des matières

1	Analyse préliminaire	3
1.1	Introduction	3
1.2	Objectifs.....	3
1.2.1	Objectif 1 Affichage de l'heure en temps réel	4
1.2.2	Objectif 2 Mesure et affichage des données environnementales.....	4
1.2.3	Objectif 3 Alerte visuelle en cas de mauvaise qualité de l'air.....	4
1.2.4	Objectif 4 Interaction utilisateur via les boutons poussoirs	4
1.2.5	Objectif 5 Qualité et maintenabilité du code.....	4
1.2.6	Objectif 6 Documentation complète	4
1.3	Analyse des besoins matériels et logiciels.....	4
1.3.1	Besoins matériels	5
1.3.2	Besoins logiciels	8
1.4	Analyse des risques.....	10
1.4.1	Risque 1 Erreur de câblage ou de brasure	10
1.4.2	Risque 2 Composant défectueux ou endommagé.....	11
1.4.3	Risque 3 Conflit ou incompatibilité entre les bibliothèques	11
1.4.4	Risque 4 Dépassement du budget temps.....	11
1.4.5	Risque 5 Problème de communication I ² C entre les composants.....	12
1.4.6	Risque 6 Perte de données ou corruption du code source	12
1.4.7	Risque 7 Difficultés avec l'environnement CLion / PlatformIO	12
1.5	Planification initiale	14
2	Analyse.....	20
3	Conception	20
3.1	Concept	20

3.2	Stratégie de test.....	20
3.3	Risques techniques	21
3.4	Planification	21
3.5	Dossier de conception	21
4	Réalisation.....	22
4.1	Dossier de réalisation	22
4.2	Description des tests effectués	22
4.3	Erreurs restantes	23
4.4	Liste des documents fournis	23
5	Conclusions	24
6	Annexes.....	25
6.1	Résumé du rapport du TPI / version succincte de la documentation	25
6.2	Sources – Bibliographie.....	25
6.3	Journal de travail	25
6.4	Manuel d'Installation	25
6.5	Manuel d'Utilisation.....	25
6.6	Archives du projet.....	25

1 Analyse préliminaire

1.1 Introduction

Ce projet s'inscrit dans le cadre du Travail de Projet Individuel (TPI) de la formation d'Informaticien CFC (Ordonnance 2021), orientation Exploitation et infrastructure, réalisé au CPNV à Sainte-Croix. Il porte sur la conception et la réalisation d'une horloge murale à LED avec capteurs environnementaux, pilotée par une carte Arduino.

L'idée de départ part d'un constat concret : de nombreuses salles de classe du CPNV sont équipées de petits boîtiers mesurant le taux de CO₂ dans l'air, afin de surveiller la qualité de l'environnement. Ces appareils effectuent leur travail, mais ils souffrent d'un défaut important : ils n'émettent aucune alerte visuelle ou sonore lorsque le taux devient problématique. Résultat, les personnes présentes dans la salle doivent penser à regarder l'appareil régulièrement pour savoir si l'air est encore acceptable ce qui, en pratique, n'arrive pas toujours.

Ce projet vise à pallier ce manque en proposant un appareil qui combine deux fonctions en une : d'un côté, une horloge murale affichant l'heure en temps réel grâce à un anneau de LED NeoPixel animé, et de l'autre, un système de surveillance environnementale affichant en continu la température, l'humidité, la pression atmosphérique et la qualité de l'air (COV) sur un écran alphanumérique. Dès que la qualité de l'air se dégrade, l'anneau LED change de comportement pour émettre une alerte visuelle immédiate et impossible à ignorer.

L'appareil repose sur plusieurs composants matériels mis à disposition : une carte Arduino REV3, quatre anneaux Adafruit NeoPixel formant un cercle de 60 LED, un capteur environnemental BME680, un module d'horloge temps réel RTC DS1307 garantissant la précision de l'heure même en cas de coupure de courant, un affichage Adafruit 4×14 segments, ainsi que six boutons poussoirs permettant à l'utilisateur de régler l'heure, d'ajuster la luminosité et de changer le mode d'affichage.

Ce projet représente une réelle opportunité d'apprentissage dans le domaine des systèmes embarqués, en combinant des aspects de programmation bas niveau, de câblage électronique, de gestion de capteurs et de conception d'une interface utilisateur simple mais efficace. Il s'appuie sur des compétences acquises notamment dans les modules INI-03, MA-20, 319, ainsi que sur le projet Système Embarqué en Arduino déjà réalisé en formation.

Aucun travail préliminaire spécifique à ce projet n'avait été effectué avant le début du TPI. L'intégralité de la conception, du câblage et du développement est réalisée durant la période du 23 avril au 21 mai 2026, dans les locaux du CPNV.

1.2 Objectifs

Les objectifs suivants ont été définis sur la base du cahier des charges. Chacun d'eux devra pouvoir être vérifié et validé à la fin du projet, que ce soit par démonstration directe, par les résultats des tests ou par la lecture du rapport.

1.2.1 Objectif 1 Affichage de l'heure en temps réel

L'horloge doit afficher l'heure courante (heures, minutes, secondes) de manière fluide et lisible sur l'anneau de 60 LED NeoPixel, composé de quatre quarts d'anneau assemblés. L'affichage doit rester correct même après une coupure de courant, grâce au module RTC DS1307.

1.2.2 Objectif 2 Mesure et affichage des données environnementales

Le système doit lire en continu les données du capteur BME680 température, humidité, pression atmosphérique et qualité de l'air (COV) et les afficher de manière alternée et claire sur l'écran alphanumérique 4×14 segments.

1.2.3 Objectif 3 Alerte visuelle en cas de mauvaise qualité de l'air

Lorsque le niveau de COV dépasse un seuil défini, l'anneau de LED doit réagir visuellement de façon distincte (changement de couleur, clignotement) afin d'alerter immédiatement les personnes présentes dans la salle, sans qu'elles aient besoin de regarder l'écran.

1.2.4 Objectif 4 Interaction utilisateur via les boutons poussoirs

Les six boutons poussoirs doivent permettre à l'utilisateur de régler manuellement l'heure, de changer le mode d'affichage de l'écran alphanumérique et d'ajuster la luminosité des LED, avec une gestion correcte du rebond (debounce).

1.2.5 Objectif 5 Qualité et maintenabilité du code

Le code Arduino doit être structuré en modules distincts (gestion RTC, NeoPixel, BME680, HT16K33, boutons), commenté et versionné sur un dépôt GitHub mis à jour quotidiennement.

1.2.6 Objectif 6 Documentation complète

Un rapport de projet complet, un journal de travail, une procédure d'installation et de mise en service, ainsi qu'une archive du projet doit être livrés dans les délais définis par le cahier des charges.

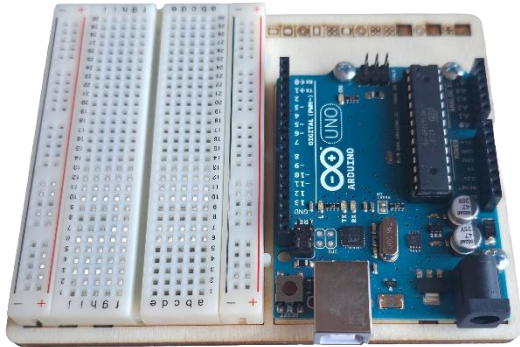
1.3 Analyse des besoins matériels et logiciels

Avant de commencer toute phase de conception ou de réalisation, il est essentiel d'identifier et de comprendre l'ensemble des ressources nécessaires à la réalisation du projet. Cette section passe en revue le matériel mis à disposition ainsi que les outils logiciels qui seront utilisés tout au long du développement.

1.3.1 Besoins matériels

1.3.1.1 Carte Arduino REV3

La carte Arduino REV3 constitue le cœur du projet. C'est elle qui orchestre l'ensemble du système : elle lit les données des capteurs, calcule l'affichage de l'heure, pilote les LED et réagit aux actions de l'utilisateur sur les boutons. Elle communique avec les différents composants via les protocoles I²C et les broches numériques/analogiques disponibles.



1.3.1.2 Anneaux Adafruit NeoPixel 1/4 — 60 LED au total x4

Ces quatre quarts d'anneau, assemblés pour former un cercle complet de 60 LED RGB, constituent l'affichage principal de l'horloge. Chaque LED est adressable individuellement, ce qui permet de créer des animations fluides pour représenter les heures, les minutes et les secondes, mais aussi de changer l'apparence de l'anneau en cas d'alerte environnementale. Leur pilotage se fait via une seule broche de données et la bibliothèque Adafruit NeoPixel.



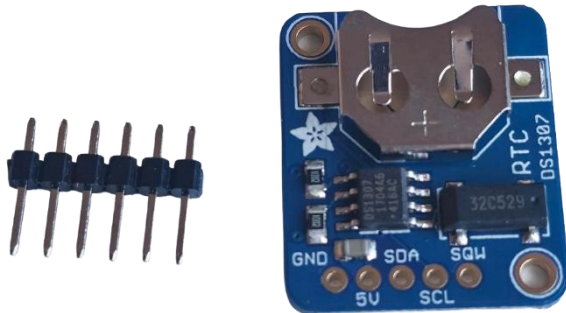
1.3.1.3 Capteur environnemental BME680

Ce capteur multifonction est capable de mesurer simultanément la température ambiante, l'humidité relative, la pression atmosphérique et la qualité de l'air via un indice de composés organiques volatils (COV). Il communique avec l'Arduino via le bus I²C et constitue la source de toutes les données environnementales affichées et surveillées par le système.



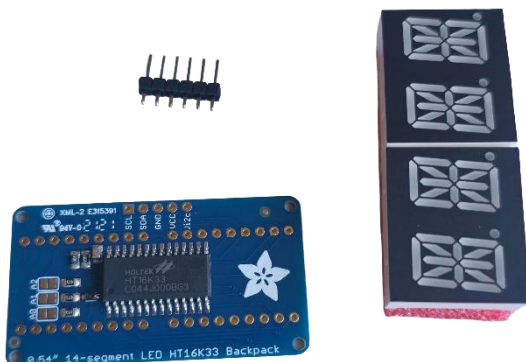
1.3.1.4 Module horloge temps réel RTC DS1307

Le module RTC DS1307 est une horloge temps réel qui maintient l'heure exacte de manière autonome, même en cas de coupure de courant, grâce à une pile de sauvegarde intégrée. Il communique également via I²C et garantit que l'horloge affichée reste toujours précise, sans dérive, tout au long de l'utilisation de l'appareil.



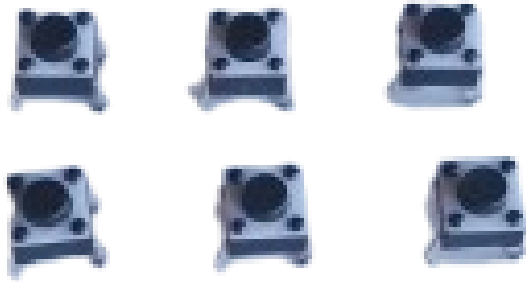
1.3.1.5 Affichage Adafruit 4×14 segments alphanumérique HT16K33

Cet affichage alphanumérique, basé sur le contrôleur HT16K33 et pilotable en I²C, permet d'afficher du texte et des chiffres. Il est utilisé pour présenter en alternance les différentes mesures environnementales (température, humidité, pression, qualité de l'air) de façon lisible et compacte.



1.3.1.6 Boutons poussoirs x6

Les six boutons permettent à l'utilisateur d'interagir directement avec le système. Ils servent à régler manuellement l'heure, à changer le mode d'affichage de l'écran alphanumérique, et à ajuster la luminosité des LED. Une gestion du rebond (debounce) devra être implémentée pour garantir un comportement fiable à chaque pression.



1.3.1.7 Poste de brasure et composants électroniques associés

Le projet nécessite un travail de câblage et de brasure pour assembler les différents composants de manière solide et durable. Des résistances, condensateurs, fils de connexion et une breadboard sont mis à disposition pour réaliser le montage électronique.

1.3.1.8 Cable USB-B 2.0 vers USB-A 2.0

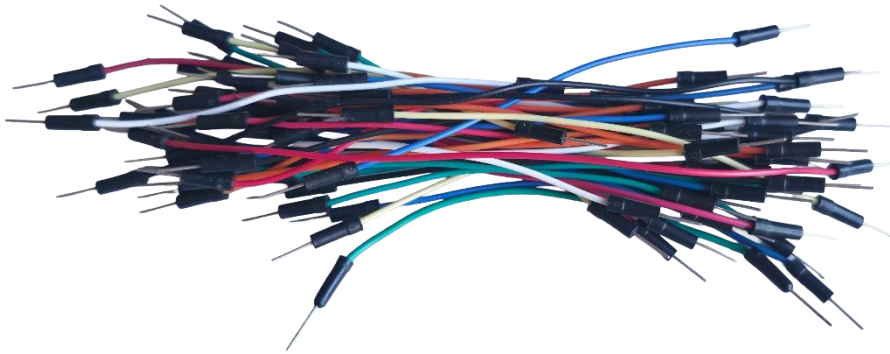
Ce câble est indispensable pour connecter la carte Arduino à l'ordinateur. Il sert à la fois à alimenter la carte durant le développement et à téléverser le programme compilé depuis CLion/PlatformIO directement sur le microcontrôleur. Sans ce câble, aucun transfert de code n'est possible entre l'environnement de développement et la carte.



1.3.1.9 Cable de connexion pin à pin mâle

Ces fils de connexion, aussi appelés jumpers, sont utilisés pour relier les différents composants électroniques entre eux sur la breadboard et vers les broches de l'Arduino. Ils permettent de créer le câblage de manière propre, flexible et modifiable.

sans brasure, ce qui est particulièrement utile durant la phase de prototypage et de tests.



1.3.1.10 Ordinateur du CPNV

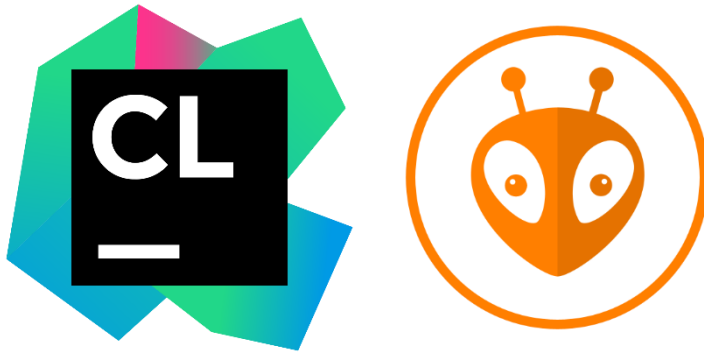
L'ordinateur mis à disposition par le CPNV constitue la station de travail principale pour l'ensemble du projet. Il héberge l'environnement de développement CLion avec PlatformIO, les outils de versioning Git, la suite Microsoft Office pour la rédaction des documents, ainsi que tous les fichiers du projet. C'est depuis cet ordinateur que le code est écrit, compilé et téléversé sur la carte Arduino tout au long de la période de réalisation.



1.3.2 Besoins logiciels

1.3.2.1 CLion avec l'extension PlatformIO

Pour le développement du code embarqué, j'ai choisi d'utiliser CLion, l'IDE de JetBrains, couplé à l'extension PlatformIO. Ce choix s'écarte de l'IDE Arduino classique, mais offre des avantages significatifs pour un projet de cette envergure : un éditeur de code puissant avec autocomplétion intelligente, une gestion propre des bibliothèques via PlatformIO, un débogage facilité, et une meilleure organisation du projet en fichiers et modules distincts. PlatformIO prend en charge nativement les cartes Arduino et gère automatiquement les dépendances, ce qui simplifie l'intégration des bibliothèques tierces.



1.3.2.2 Bibliothèque Adafruit NeoPixel

Cette bibliothèque officielle d'Adafruit est indispensable pour piloter les anneaux LED. Elle permet de contrôler chaque LED individuellement, de définir des couleurs en RGB et de créer des animations. Elle sera intégrée au projet via le gestionnaire de bibliothèques de PlatformIO.

1.3.2.3 Bibliothèque Adafruit BME680

La bibliothèque Adafruit pour le capteur BME680 fournit une interface simple et fiable pour lire toutes les données du capteur (température, humidité, pression, COV) sans avoir à gérer manuellement les registres I²C de bas niveau.

1.3.2.4 Bibliothèque RTCLib

La bibliothèque RTCLib, développée par Adafruit, facilite la communication avec le module RTC DS1307. Elle permet de lire et d'écrire l'heure courante de façon claire, et d'effectuer des conversions de dates et d'heures sans complexité inutile.

1.3.2.5 Bibliothèque Adafruit LEDBackpack / GFX

Ces bibliothèques permettent de piloter l'affichage alphanumérique HT16K33 via I²C. Elles offrent des fonctions simples pour écrire du texte ou des valeurs numériques sur les segments, ce qui rend l'affichage des données environnementales straightforward à implémenter.

1.3.2.6 Git et GitHub

L'ensemble du code source sera versionné avec Git et publié sur un dépôt GitHub mis à jour quotidiennement, conformément aux exigences du cahier des charges. Cela garantit un suivi précis de l'évolution du projet, permet de revenir à une version antérieure en cas de problème, et constitue une archive complète du travail effectué.



1.3.2.7 Suite Microsoft Office

Microsoft Word sera utilisé pour la rédaction du rapport de projet et du journal de travail, conformément aux livrables attendus. Microsoft PowerPoint pourra être utilisé pour préparer la présentation orale prévue les 2 ou 3 juin 2026.



1.4 Analyse des risques

Avant de se lancer dans la phase de conception et de réalisation, il est important d'identifier les risques potentiels qui pourraient compromettre le bon déroulement du projet. Cette analyse permet d'anticiper les problèmes, de préparer des solutions de repli et d'aborder chaque étape avec plus de sérénité.

Voici la section complète sur l'analyse des risques, en intégrant tes inquiétudes sur la brasure et les branchements.

1.4.1 Risque 1 Erreur de câblage ou de brasure

Niveau de risque : Élevé

C'est le risque qui m'inquiète le plus dans ce projet. Le montage implique de nombreux composants reliés entre eux via des câbles, des broches et des points de brasure. Une erreur de câblage — comme inverser l'alimentation et la masse (GND/VCC) — ou une mauvaise brasure — comme un faux contact ou un court-circuit — peut endommager irrémédiablement un composant, voire la carte Arduino elle-même.

Mesures préventives :

Avant chaque branchement, je vérifierai systématiquement le schéma de câblage. Je procéderai composant par composant, en testant le bon fonctionnement de chacun avant de passer au suivant. Pour la brasure, je m'assurerai que les joints sont propres, brillants et bien formés, et j'utiliserai un multimètre pour vérifier la continuité électrique et l'absence de court-circuit avant toute mise sous tension.

1.4.2 Risque 2 Composant défectueux ou endommagé

Niveau de risque : Moyen

Malgré toutes les précautions prises, un composant peut s'avérer défectueux dès le départ ou être endommagé lors du montage (surchauffe à la brasure, électricité statique, mauvaise manipulation). Si un module clé comme le BME680 ou le RTC ne fonctionne pas, cela peut bloquer une partie importante du développement.

Mesures préventives :

Je testerai chaque composant individuellement dès son branchement, à l'aide de programmes de test simples, avant de les intégrer dans le projet global. En cas de panne avérée, j'en informerai immédiatement mon chef de projet afin d'obtenir un composant de remplacement dans les meilleurs délais.

1.4.3 Risque 3 Conflit ou incompatibilité entre les bibliothèques

Niveau de risque : Moyen

Le projet utilise plusieurs bibliothèques tierces simultanément : Adafruit NeoPixel, Adafruit BME680, RTCLib et Adafruit LEDBackpack. Il est possible que certaines versions de ces bibliothèques entrent en conflit, consomment trop de mémoire RAM ou Flash sur l'Arduino, ou provoquent des comportements inattendus lorsqu'elles sont utilisées ensemble.

Mesures préventives :

J'intégrerai les bibliothèques progressivement, en testant la compatibilité à chaque ajout. Je consulterai la documentation officielle d'Adafruit et les issues GitHub des bibliothèques concernées pour m'assurer d'utiliser des versions stables et compatibles. En cas de problème de mémoire, j'optimiserai le code en réduisant l'utilisation des variables globales et en privilégiant les types de données les plus légers.

1.4.4 Risque 4 Dépassement du budget temps

Niveau de risque : Moyen

Avec 90 heures disponibles et un projet qui touche à la fois au matériel et au logiciel, le risque de prendre du retard est réel. Certaines tâches peuvent s'avérer plus longues que prévu notamment le débogage, l'intégration des modules ou la rédaction du rapport et empiéter sur les étapes suivantes.

Mesures préventives :

Je suivrai rigoureusement ma planification initiale et mettrai à jour mon journal de travail quotidiennement pour détecter tout écart rapidement. Si une tâche prend trop de temps, je l'évaluerai et déciderai si je peux la simplifier ou la reporter, en priorisant toujours les fonctionnalités essentielles (affichage de l'heure, lecture des capteurs) avant les fonctionnalités secondaires (animations, modes d'affichage avancés).

1.4.5 Risque 5 Problème de communication I²C entre les composants

Niveau de risque : Moyen

Trois composants du projet (BME680, RTC DS1307 et HT16K33) communiquent tous via le bus I²C. Si deux composants partagent la même adresse I²C ou si le bus est mal configuré, les communications peuvent échouer ou se mélanger, rendant les données illisibles.

Mesures préventives :

Je vérifierai en amont les adresses I²C de chaque composant et m'assurerai qu'elles sont toutes différentes. J'utiliserai un scanner I²C (programme Arduino simple) pour détecter tous les appareils connectés sur le bus et confirmer qu'ils sont bien reconnus avant d'aller plus loin dans le développement.

1.4.6 Risque 6 Perte de données ou corruption du code source

Niveau de risque : Faible

Une panne informatique, une fausse manipulation ou un problème de synchronisation pourrait entraîner la perte d'une partie du code ou des documents rédigés, avec un impact potentiellement important sur l'avancement du projet.

Mesures préventives :

Le code sera versionné et poussé sur GitHub quotidiennement, conformément aux exigences du cahier des charges. Les documents seront sauvegardés régulièrement sur le réseau du CPNV. Ces deux pratiques combinées réduisent ce risque à un niveau très faible.

1.4.7 Risque 7 Difficultés avec l'environnement CLion / PlatformIO

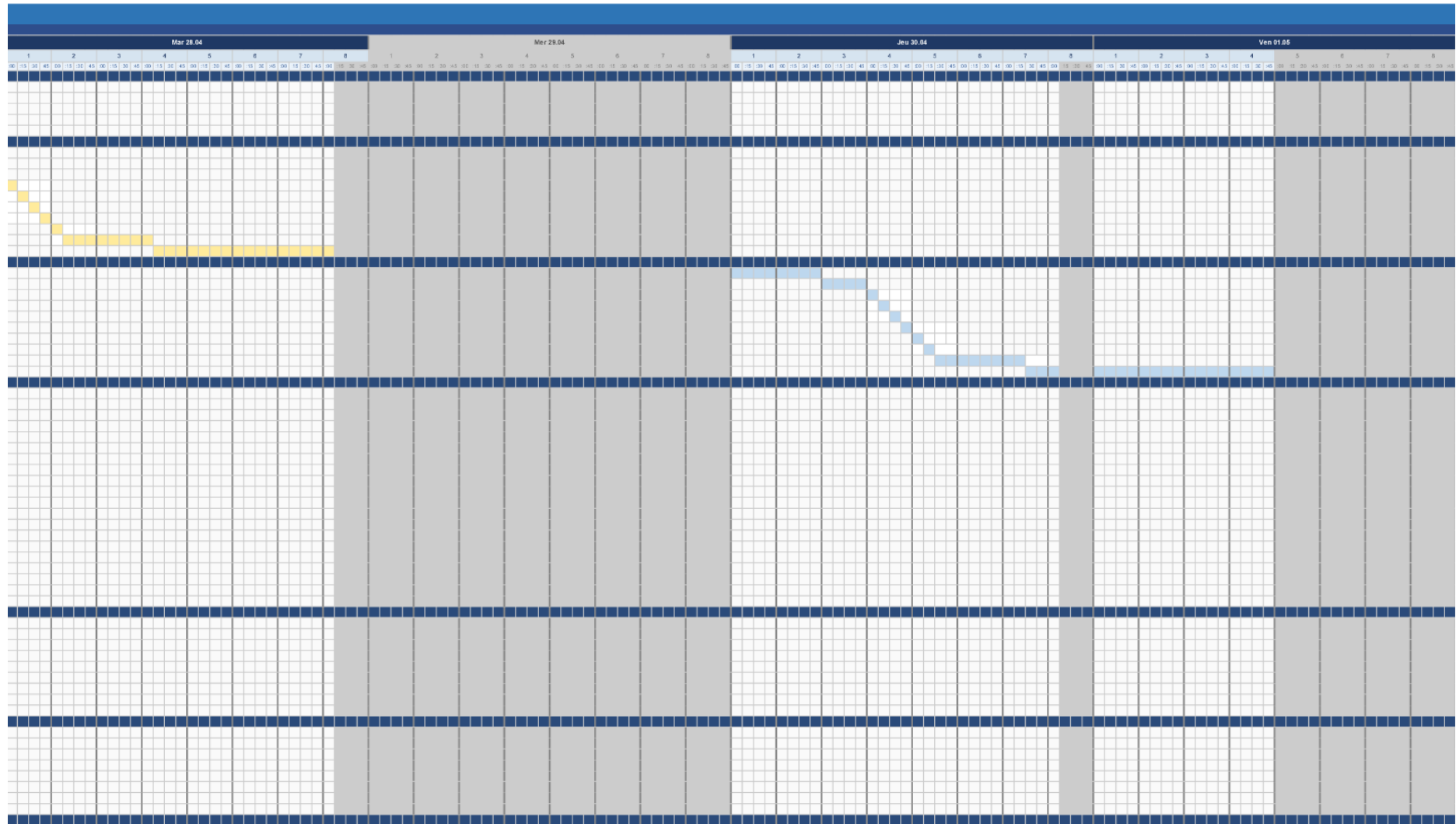
Niveau de risque : Faible

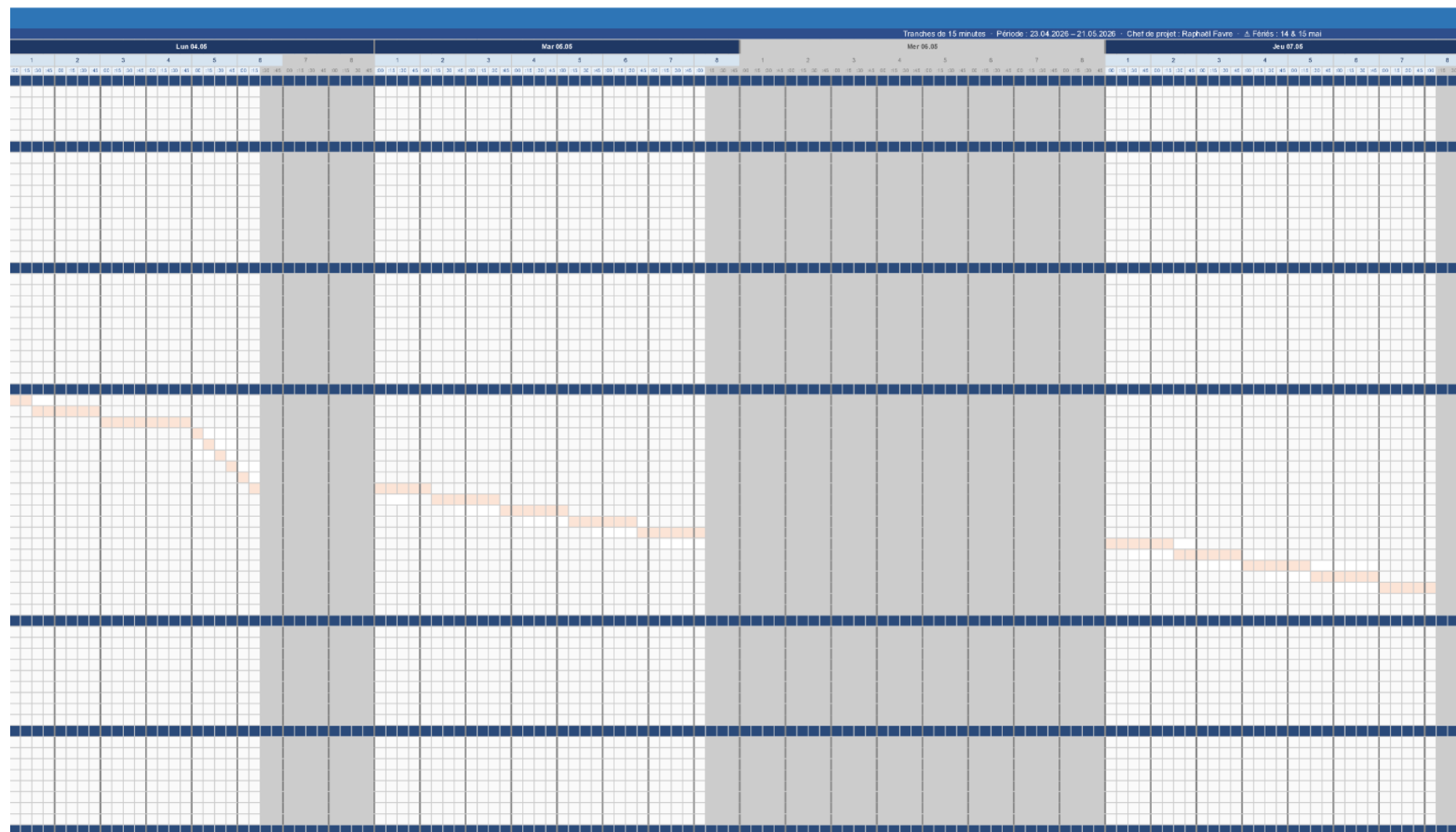
CLion couplé à PlatformIO est un environnement plus avancé que l'IDE Arduino standard. Une configuration incorrecte, un problème de détection du port série ou une mise à jour inattendue pourrait ralentir le démarrage du développement.

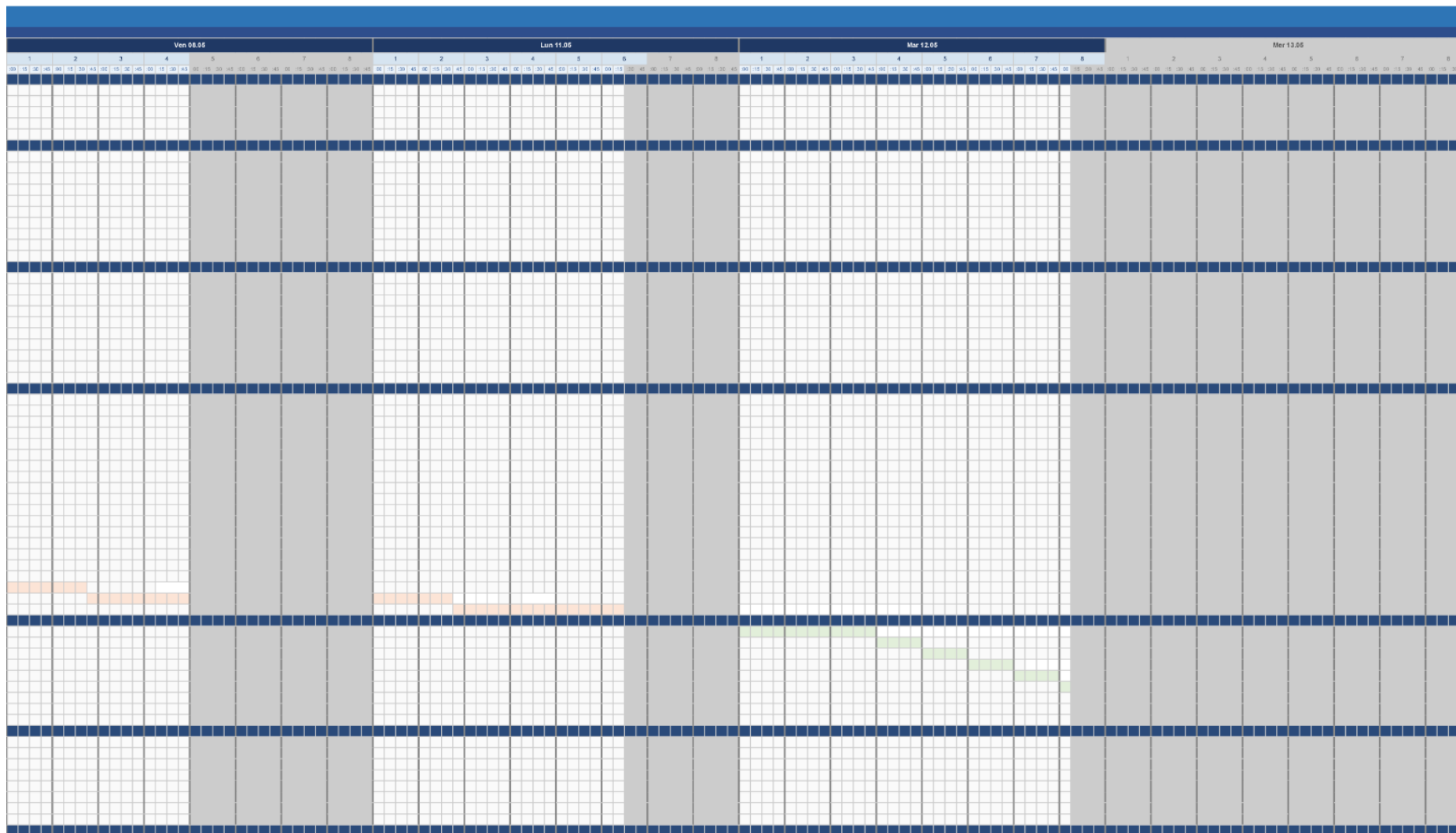
Mesures préventives :

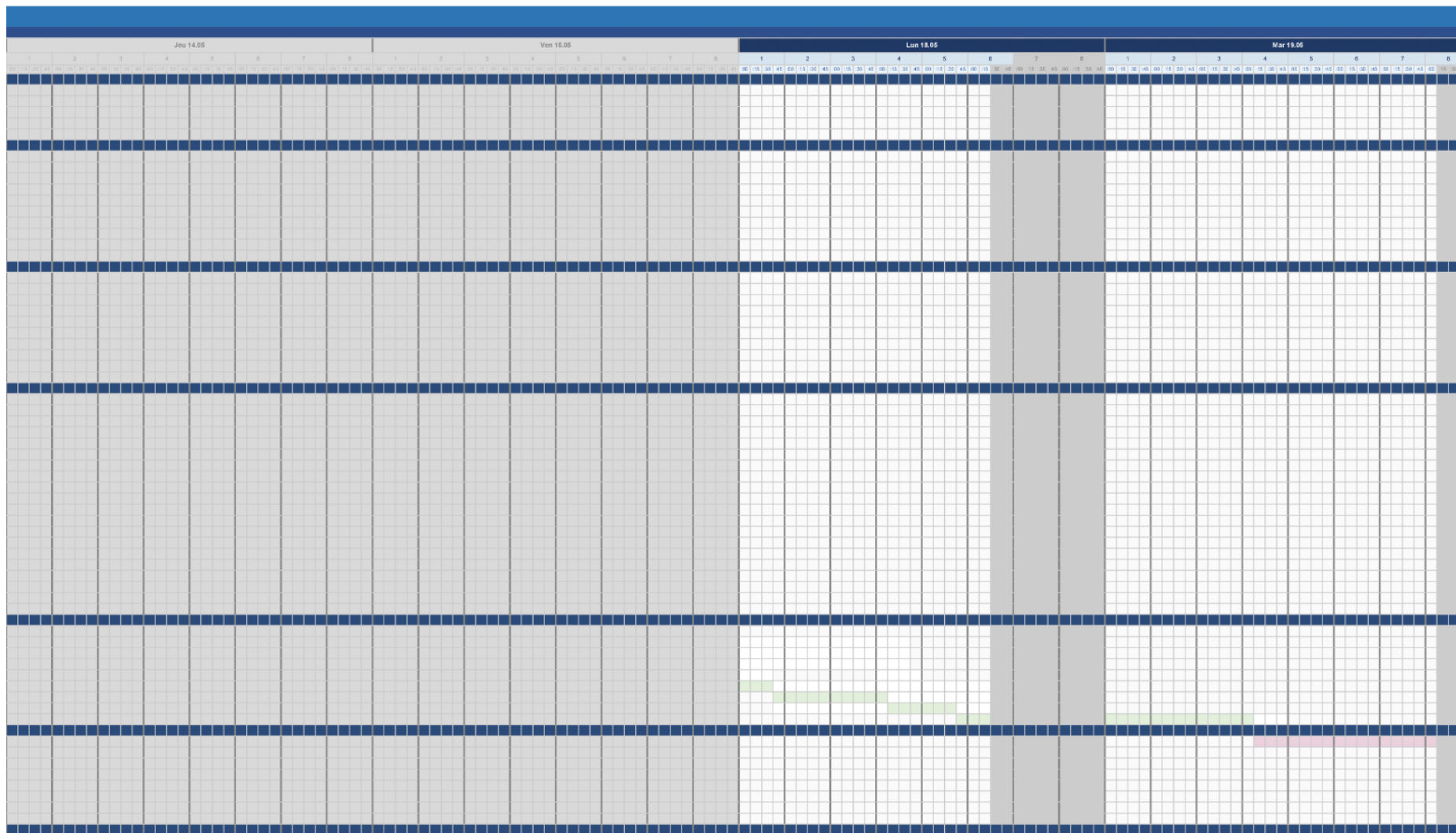
Je consacrerai le temps nécessaire à la mise en place et à la vérification de l'environnement de développement dès le premier jour, avant toute autre tâche de programmation. Si des difficultés persistantes apparaissent, je pourrai me rabattre temporairement sur l'IDE Arduino classique pour ne pas bloquer l'avancement, puis revenir à CLion une fois le problème résolu.

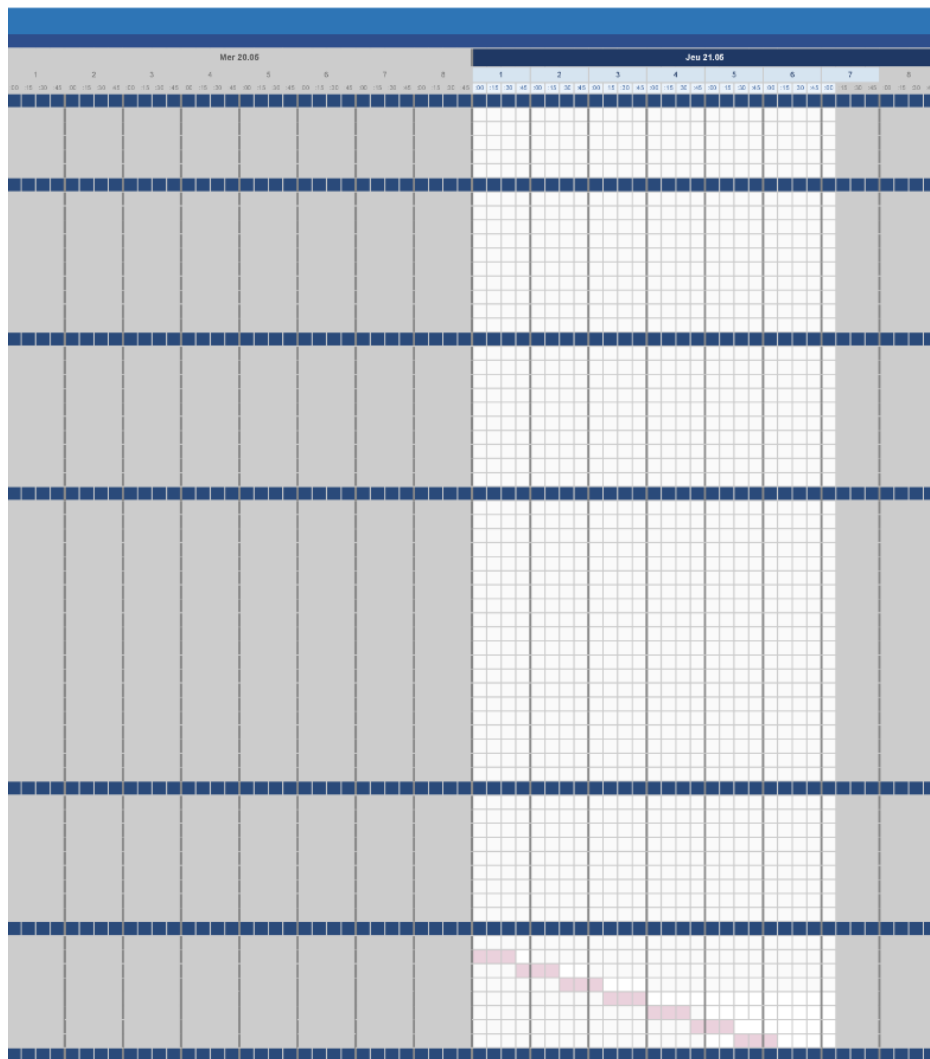
N°	Titre de la tâche	H prev.	H real.
Jeu 23.04			
Ven 24.04			
Lun 27.04			
1	Analyse préliminaire	11.25h	
1.1	Lecture du cahier des charges	1.50h	1.50h
1.2	Planification initiale	5.75h	4.50h
1.3	Analyse des besoins matériels et logiciel	2.00h	
1.4	Vérification du matériel disponible	0.50h	
1.5	Analyse des risques	1.50h	
2	Analyse	11.75h	
2.1	Analyse de l'IDE à utiliser	1.50h	
2.2	Analyse des plugins et bibliothèque	1.50h	
2.3	Analyse de la gestion de chaque élément matériel	1.50h	
2.4	Etude du module RTC DS1307 (datasheet, I2C)	0.25h	
2.5	Etude des anneaux NeoPixel 1/4 (lib Adafruit)	0.25h	
2.6	Etude du capteur BME680 (temp., hum., pression, COV ₂)	0.25h	
2.7	Etude de l'affichage HT16K33 4x14-segments	0.25h	
2.8	Analyse des 6 boutons poussoirs (debounce, logique)	0.25h	
2.9	Identification des contraintes techniques	2.00h	
2.10	Rédaction rapport — section Analyse	4.00h	
3	Conception	11.25h	
3.1	Schéma de câblage complet (Fritzing ou équivalent)	2.00h	
3.2	Définition de l'architecture logicielle (modules)	1.00h	
3.3	Conception du module gestion RTC	0.25h	
3.4	Conception du module affichage NeoPixel	0.25h	
3.5	Conception du module capteur BME680	0.25h	
3.6	Conception du module affichage HT16K33	0.25h	
3.7	Conception de la logique des boutons poussoirs	0.25h	
3.8	Conception du système d'alerte visuelle (COV)	0.25h	
3.9	Diagramme de flux	2.00h	
3.10	Rédaction rapport — section Conception	4.75h	
4	Réalisation	29.50h	
4.1	Mise en place du projet Arduino & dépôt GitHub	0.50h	
4.2	Montage du breadboard principal	1.50h	
4.3	Brasure des connecteurs & composants fixes	2.00h	
4.4	Câblage des 4 anneaux NeoPixel	0.25h	
4.5	Câblage du module RTC DS1307	0.25h	
4.6	Câblage du capteur BME680	0.25h	
4.7	Câblage de l'affichage HT16K33	0.25h	
4.8	Câblage des 6 boutons poussoirs	0.25h	
4.9	Programmation : lecture & sync RTC DS1307	1.50h	
4.1	Programmation : affichage heure sur NeoPixel (animation fluide)	1.50h	
4.11	Programmation : lecture capteur BME680	1.50h	
4.12	Programmation : affichage données env. sur HT16K33 (mode alterné)	1.50h	
4.13	Programmation : gestion bouton réglage heure	1.50h	
4.14	Programmation : bouton changement mode affichage	1.50h	
4.15	Programmation : bouton ajustement luminosité LED	1.50h	
4.16	Programmation : détection qualité air (seuils COV ₂)	1.50h	
4.17	Programmation : alerte visuelle LED (clignotement couleur)	1.50h	
4.18	Intégration & assemblage global de tous les modules	3.00h	
4.19	Débogage & corrections post-intégration	4.00h	
4.20	Rédaction rapport — section Réalisation	3.75h	
5	Tests	14.00h	
5.1	Rédaction du plan de tests	3.00h	
5.2	Tests unitaires : module RTC (précision heure)	1.00h	
5.3	Tests unitaires : module NeoPixel (affichage, anim.)	1.00h	
5.4	Tests unitaires : capteur BME680 (valeurs plausibles)	1.00h	
5.5	Tests unitaires : affichage HT16K33	1.00h	
5.6	Tests unitaires : boutons poussoirs (debounce, modes)	1.00h	
5.7	Tests d'intégration globale du système	2.50h	
5.8	Tests de l'alerte visuelle qualité de l'air	1.50h	
5.9	Rédaction rapport — section Tests & résultats	4.00h	
6	Finalisation	9.25h	
6.1	Correction des défauts identifiés lors des tests	4.00h	
6.2	Finalisation & mise en forme du rapport complet	0.75h	
6.3	Rédaction de la procédure d'installation & mise en service	0.75h	
6.4	dernière vérification du journal de travail complet	0.75h	
6.5	Archivage final sur GitHub (commit & tag release)	0.75h	
6.6	Export PDF du rapport et du journal	0.75h	
6.7	Vérification de la complétude des livrables	0.75h	
6.8	Envoi du dossier aux experts & chef de projet	0.75h	
7	Total	89.00h	











2 Analyse

3 Conception

3.1 Concept

Le concept complet avec toutes ses annexes:

Par exemple :

- *Multimédia: carte de site, maquettes papier, story board préliminaire, ...*
- *Bases de données: interfaces graphiques, modèle conceptuel.*
- *Programmation: interfaces graphiques, maquettes, analyse fonctionnelle...*
- *...*

3.2 Stratégie de test

Décrire la stratégie globale de test:

- *types de des tests et ordre dans lequel ils seront effectués.*
- *les moyens à mettre en œuvre.*
- *couverture des tests (tests exhaustifs ou non, si non, pourquoi ?).*
- *données de test à prévoir (données réelles ?).*
- *les testeurs extérieurs éventuels.*

3.3 Risques techniques

- *risques techniques (complexité, manque de compétences, ...).*

Décrire aussi quelles solutions ont été appliquées pour réduire les risques (priorités, formation, actions, ...).

3.4 Planification

Révision de la planification initiale du projet :

- *planning indiquant les dates de début et de fin du projet ainsi que le découpage connu des diverses phases.*
- *partage des tâches en cas de travail à plusieurs.*

*Il s'agit en principe de la planification **définitive du projet**. Elle peut être ensuite affinée (découpage des tâches). Si les délais doivent être ensuite modifiés, le responsable de projet doit être avisé, et les raisons doivent être expliquées dans l'historique.*

3.5 Dossier de conception

Fournir tous les document de conception:

- *le choix du matériel HW*
- *le choix des systèmes d'exploitation pour la réalisation et l'utilisation*
- *le choix des outils logiciels pour la réalisation et l'utilisation*

- *site web: réaliser les maquettes avec un logiciel, décrire toutes les animations sur papier, définir les mots-clés, choisir une formule d'hébergement, définir la méthode de mise à jour, ...*
- *bases de données: décrire le modèle relationnel, le contenu détaillé des tables (caractéristiques de chaque champs) et les requêtes.*
- *programmation et scripts: organigramme, architecture du programme, découpage modulaire, entrées-sorties des modules, pseudo-code / structogramme...*

Le dossier de conception devrait permettre de sous-traiter la réalisation du projet !

4 Réalisation

4.1 Dossier de réalisation

Décrire la réalisation "physique" de votre projet

- *les répertoires où le logiciel est installé*
- *la liste de tous les fichiers et une rapide description de leur contenu (des noms qui parlent !)*
- *les versions des systèmes d'exploitation et des outils logiciels*
- *la description exacte du matériel*
- *le numéro de version de votre produit !*
- *programmation et scripts: bibliothèques externes, dictionnaire des données, reconstruction du logiciel - cible à partir des sources.*

NOTE : Évitez d'inclure les listings des sources, à moins que vous ne désiriez en expliquer une partie vous paraissant importante. Dans ce cas n'incluez que cette partie...

4.2 Description des tests effectués

Pour chaque partie testée de votre projet, il faut décrire:

- *les conditions exactes de chaque test*
- *les preuves de test (papier ou fichier)*
- *tests sans preuve: fournir au moins une description*

4.3 Erreurs restantes

S'il reste encore des erreurs:

- *Description détaillée*
- *Conséquences sur l'utilisation du produit*
- *Actions envisagées ou possibles*

4.4 Liste des documents fournis

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- *le rapport de projet*
- *le manuel d'Installation (en annexe)*
- *le manuel d'Utilisation avec des exemples graphiques (en annexe)*
- *autres...*

5 Conclusions

Développez en tous cas les points suivants:

- *Objectifs atteints / non-atteints*
- *Points positifs / négatifs*
- *Difficultés particulières*
- *Suites possibles pour le projet (évolutions & améliorations)*

6 Annexes

6.1 Résumé du rapport du TPI / version succincte de la documentation

6.2 Sources – Bibliographie

Liste des livres utilisés (Titre, auteur, date), des sites Internet (URL) consultés, des articles (Revue, date, titre, auteur)... Et de toutes les aides externes (noms)

6.3 Journal de travail

Date	Durée	Activité	Remarques

6.4 Manuel d'Installation

6.5 Manuel d'Utilisation

6.6 Archives du projet

Media, ... dans une fourre en plastique